Python-Based Automated Hardware Test System Simulation

Project Report

This project simulates an automated hardware test workflow using Python. It models a Device Under Test (DUT), generates measurement readings, checks them against engineering limits, and produces a structured PASS/FAIL report in CSV format together with a final summary.

1. Objective

The objective of this project is to demonstrate realistic engineering test workflow where multiple DUTs are validated automatically. The script generates voltage, current, temperature, and sensor-output readings, applies test limits, determines overall PASS/FAIL status, and saves the results in a machine-readable report.

2. Tools Used

• Python

• Command Prompt

• CSV file generation

• Microsoft Excel

• Automated test logic

• Hardware validation concepts

3. Test Parameters

Test Parameter	Acceptable Range	Unit
Voltage	4.8 to 5.2	V
Current	0.20 to 0.60	A
Temperature	≤ 70	°C
Sensor Output	0.5 to 3.0	V

4. System Block Diagram

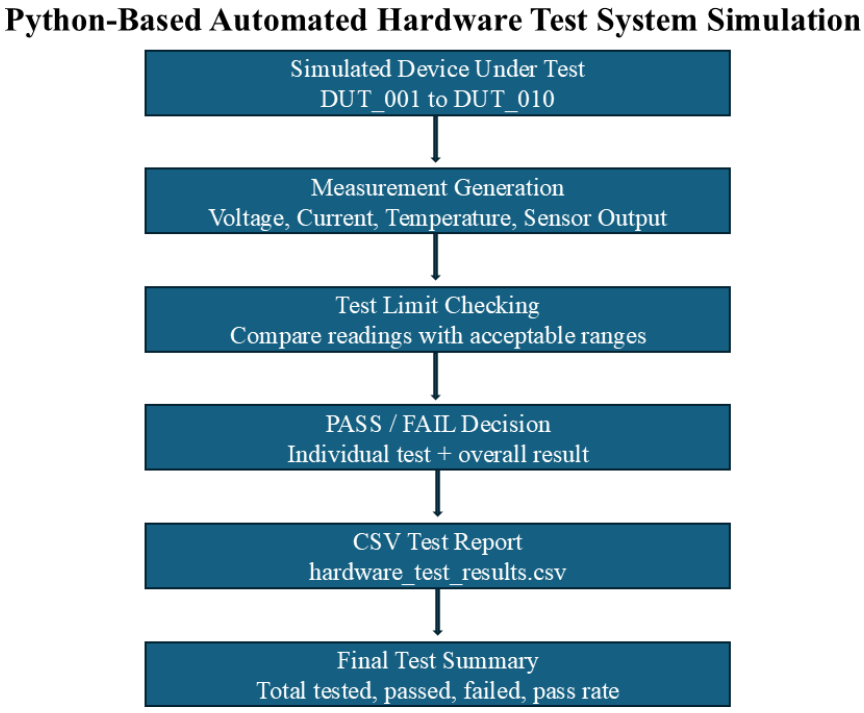


Figure 1. hardware test system block diagram.

5. Implementation

The Python script creates ten simulated DUTs (DUT_001 to DUT_010). For each DUT, it generates a voltage, current, temperature, and sensor-output reading. Each measurement is checked against predefined limits. The program then assigns PASS or FAIL to each individual test and calculates the overall result. Finally, the results are written in a CSV file, and a terminal summary is displayed.

6. Example Console Output and Summary

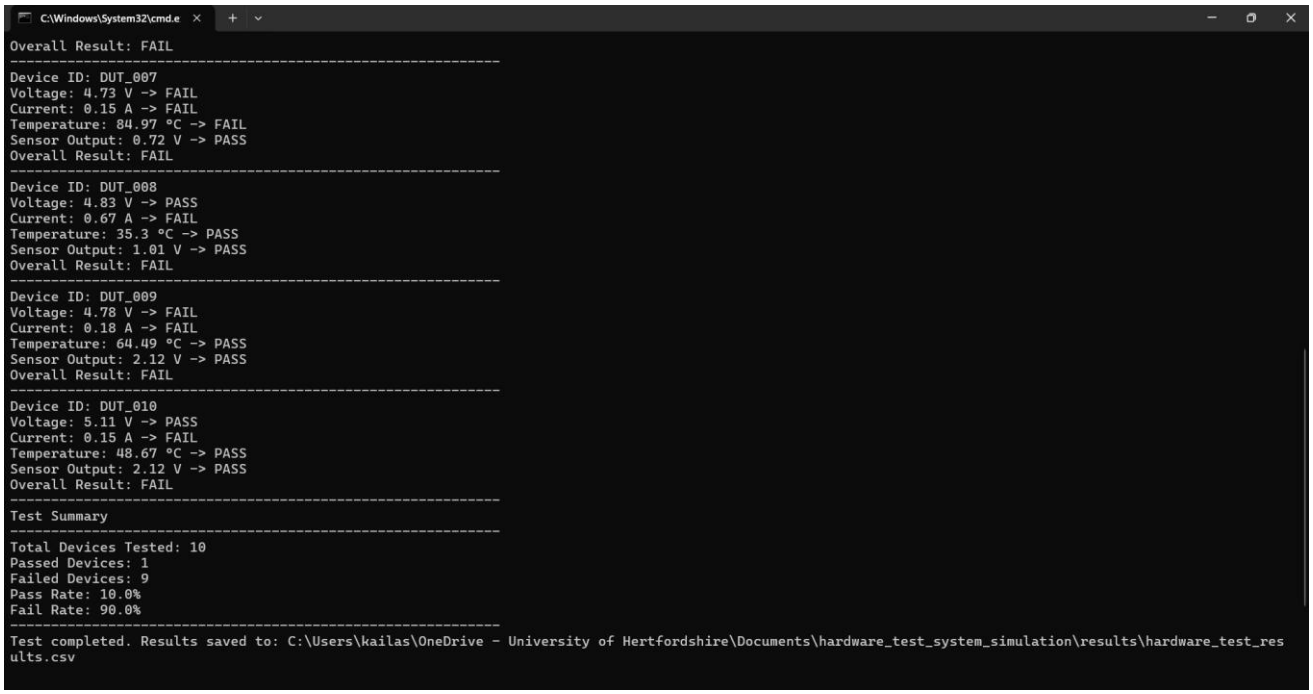
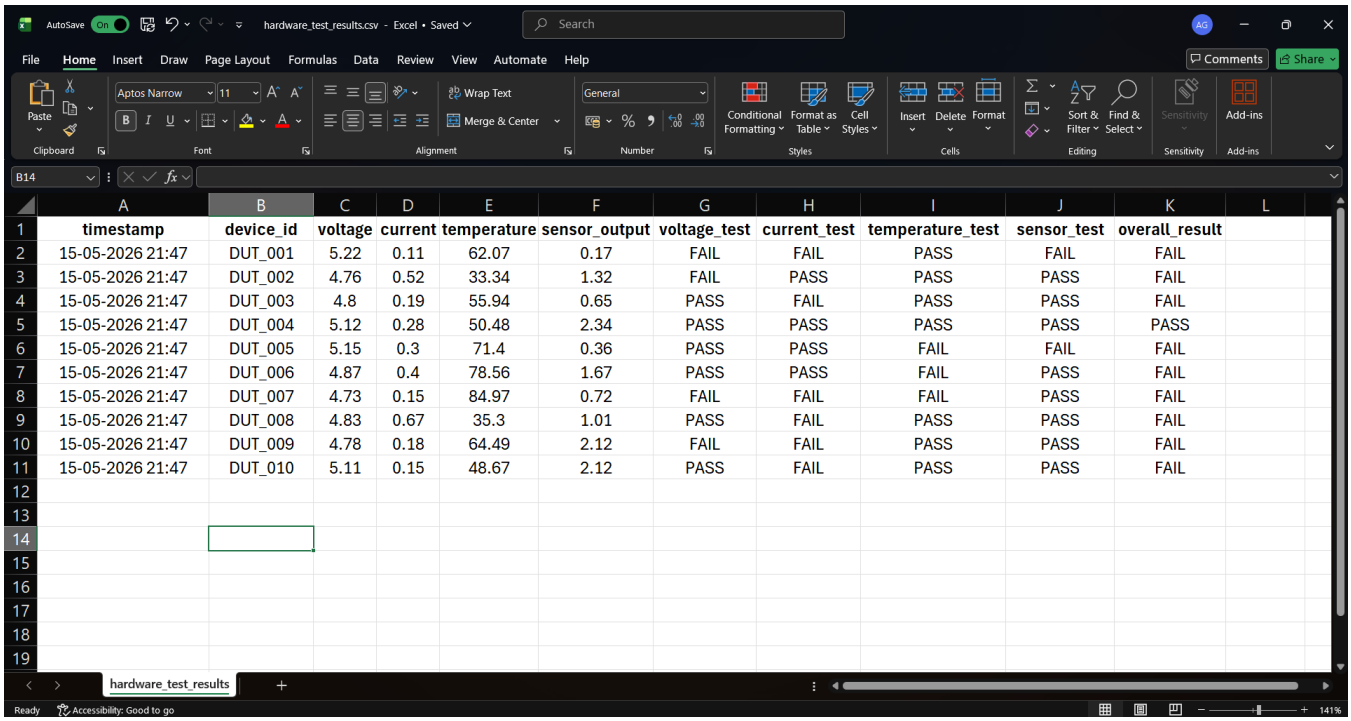


Figure 2. Console output showing individual DUT decisions and final test summary.

7. CSV Test Results

The CSV screenshot below shows the full test dataset with clear column names: timestamp, device_id, voltage, current, temperature, sensor_output, voltage_test, current_test, temperature_test,

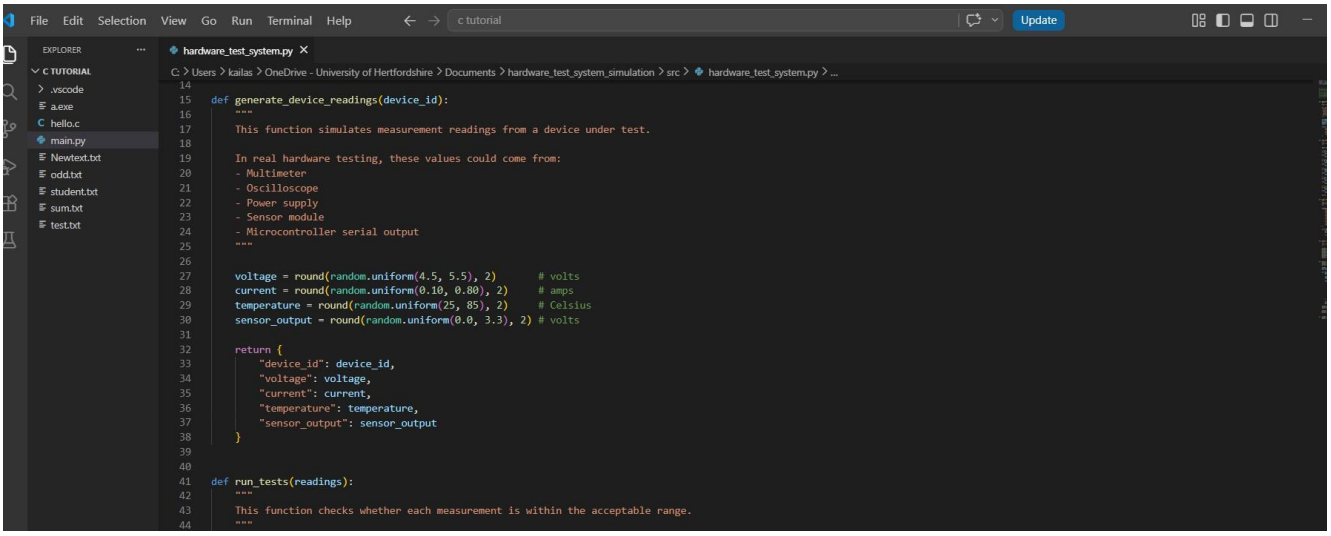
sensor_test, and overall_result.



timestamp	device_id	voltage	current	temperature	sensor_output	voltage_test	current_test	temperature_test	sensor_test	overall_result
15-05-2026 21:47	DUT_001	5.22	0.11	62.07	0.17	FAIL	FAIL	PASS	FAIL	FAIL
15-05-2026 21:47	DUT_002	4.76	0.52	33.34	1.32	FAIL	PASS	PASS	PASS	FAIL
15-05-2026 21:47	DUT_003	4.8	0.19	55.94	0.65	PASS	FAIL	PASS	PASS	FAIL
15-05-2026 21:47	DUT_004	5.12	0.28	50.48	2.34	PASS	PASS	PASS	PASS	PASS
15-05-2026 21:47	DUT_005	5.15	0.3	71.4	0.36	PASS	PASS	FAIL	FAIL	FAIL
15-05-2026 21:47	DUT_006	4.87	0.4	78.56	1.67	PASS	PASS	FAIL	PASS	FAIL
15-05-2026 21:47	DUT_007	4.73	0.15	84.97	0.72	FAIL	FAIL	FAIL	PASS	FAIL
15-05-2026 21:47	DUT_008	4.83	0.67	35.3	1.01	PASS	FAIL	PASS	PASS	FAIL
15-05-2026 21:47	DUT_009	4.78	0.18	64.49	2.12	FAIL	FAIL	PASS	PASS	FAIL
15-05-2026 21:47	DUT_010	5.11	0.15	48.67	2.12	PASS	FAIL	PASS	PASS	FAIL

Figure 3. CSV results opened in Excel.

8. Source Code Snapshot



```python
def generate_device_readings(device_id):
    """
    This function simulates measurement readings from a device under test.

    In real hardware testing, these values could come from:
    - Multimeter
    - Oscilloscope
    - Power supply
    - Sensor module
    - Microcontroller serial output
    """

    voltage = round(random.uniform(4.5, 5.5), 2)       # volts
    current = round(random.uniform(0.10, 0.80), 2)     # amps
    temperature = round(random.uniform(25, 85), 2)     # Celsius
    sensor_output = round(random.uniform(0.0, 3.3), 2) # volts

    return {
        "device_id": device_id,
        "voltage": voltage,
        "current": current,
        "temperature": temperature,
        "sensor_output": sensor_output
    }


def run_tests(readings):
    """
    This function checks whether each measurement is within the acceptable range.
    """
```

Figure 4. Source code implementing the automated hardware test workflow.

9. Observations

- Ten devices were tested automatically.
- Only one device achieved an overall PASS result in the shown run.
- Nine devices failed because one or more measured parameters were outside the acceptable limits.
- The summary reported a pass rate of 10.0% and a fail rate of 90.0%.
- This is expected because the readings are generated randomly and the limits are intentionally strict.

10. Conclusion

This project successfully demonstrates a hardware validation workflow using Python. It shows the ability to simulate test data, apply engineering limits, produce automated PASS/FAIL decisions, save structured results, and present an end-of-test summary. The project is relevant to hardware validation,

electronics testing, embedded systems, and production test engineering.

11. Future Improvements

• Add pass/fail charts and simple analytics.

• Generate a fully automatic PDF report directly from Python.

• Store test limits in a separate configuration file.

• Add fault classification for failed devices.

• Extend the workflow to support real instruments or sensor interfaces.